

Praktikumsbericht zu Versuch 6

Von Daniel Baer (3235621)

1. Klasse HashTableChaining

Python Skript: *V06_hashtable_chaining.py*

2. Prüfung der Implementierung

Python Skript: *V06_02_verification.py*

Output:

```
alpha after insert: 0.7000

slots before delete:
0: [34, 47]
1: []
2: [52]
3: []
4: []
5: [15]
6: []
.
.
.
delete 52

slots after delete:
0: [34, 47]
1: []
2: []
3: []
4: []
5: [15]
6: []
.
.
.
alpha after second insert: 0.7000
```

a) Belegungsfaktor nach dem Einfügen

Belegungsfaktor $n/m = 0.7$

b) Löschen von Eintrag 52

Die Liste, welche den Eintrag 52 enthalten hat, ist nach dem Löschen leer.

c) seq0 erneut einfügen

Der Belegungsfaktor überschreitet 1 nicht.

Nachdem die Ketten in den Slots „unbegrenzt“ sind, kann immer ein Element am Ende der Kette hinzugefügt werden. So schlägt der Verkettungsvorgang niemals fehl so lange Speicher verfügbar ist.

d) Warum kein DELETE_MARK

Da hier der Suchpfad durch das Löschen eines Elements nicht unterbrochen wird, sondern das Element direkt aus der Liste gelöscht werden kann, wird kein DELETED_MARK benötigt.

3. HashTableOpenAddressing Größe 20

Python Skript: *V06_03_HashTableOpenAddressing.py*

Output:

```
before delete: [34, UNUSED, 52, UNUSED, 50, ...
delete 52
after delete:  [34, UNUSED, DELETED, UNUSED, 50, ...

before insertion: [34, UNUSED, DELETED, UNUSED, 50, ...
insert 24, f(24) = 2
after insertion:  [34, UNUSED, 24, UNUSED, 50, ...
```

a) Löschen von Eintrag 52

Nach dem Löschen wird DELETED ausgegeben

b) Warum wird die Suche bei DELETED fortgesetzt?

DELETED bedeutet, dass hier bereits ein Eintrag in der Kette stand und es in der Kette einen Eintrag geben kann, der auf diese Zelle folgt.

UNUSED zeigt, dass diese Zelle nie genutzt wurde und so auch kein Folgeeintrag existiert. Wurde der Eintrag bisher nicht gefunden, ist er nicht in dieser Kette enthalten.

c) delete setzt Zelle auf UNUSED

Das Suchen eines Elements in der Tabelle würde vorzeitig abgebrochen werden, wenn sich das gesuchte Element nach dem gelöschten Eintrag befindet.

Beispiel:

Sei $X, Y \in \mathbb{N}$ und die Hashfunktion $h(n)$ so, dass gilt $h(X) = h(Y) = p$

Wird jetzt zuerst X und dann Y eingefügt, kommt es zu einer Kollision und Y wird an der nächsten freien oder ungenutzten Position (größer als p) nach X eingefügt.

Dann wird das Element X so gelöscht, dass die Zelle an Position p auf UNUSED gesetzt wird.

Bei einer Suche wird die Hashtabelle so weit durchlaufen, bis eine ungenutzte Zelle erreicht wird. Diese markiert das Ende der Tabelle.

Wird jetzt nach dem Eintrag Y gesucht, welcher zuvor an einer Stelle nach X eingefügt wurde, wird die Suche an der Position p beendet und die Elemente nach der Position p werden nicht mehr durchlaufen.

d) Einfügen von Eintrag 24

52 und 24 landen beide an Position 2.

24 befindet sich nach dem Einfügen auch an Position 2.

4. HashTableOpenAddressing Größe 90

Python Skript: *V06_04_TablesSeq1.py*

Output:

```
for m=90:
inserted: 87
rejected: 13
unused slots: 3
alpha: 0.9667
```

```
for m=89:
inserted: 89
rejected: 11
unused slots: 0
alpha: 1.0000
```

```
for m=20:
inserted: 77
alpha: 3.8500
```

```
probe numbers for open addressing:
alpha=0.5: 2.0000
alpha=0.6: 2.5000
alpha=0.7: 3.3333
alpha=0.8: 5.0000
alpha=0.9: 10.0000
```

a) Abgewiesene Werte

13 Werte werden bei 3 freien Slots abgewiesen, da mit der Quadratischen Sondierung bei dieser Größe nicht alle Positionen angesprochen werden.

Gemeinsame Faktoren können hier zu periodischen mustern führen, die bestimmte Hashes häufiger anspricht.

b) Größe 89

Primzahlen als Tabellengrößen verhindern periodische Muster und die Bildung Von Häufungen.

c) HashTableChaining

Es werden 77 Elemente aufgenommen, was zu einem Belegungsfaktor von 3.85 führt.

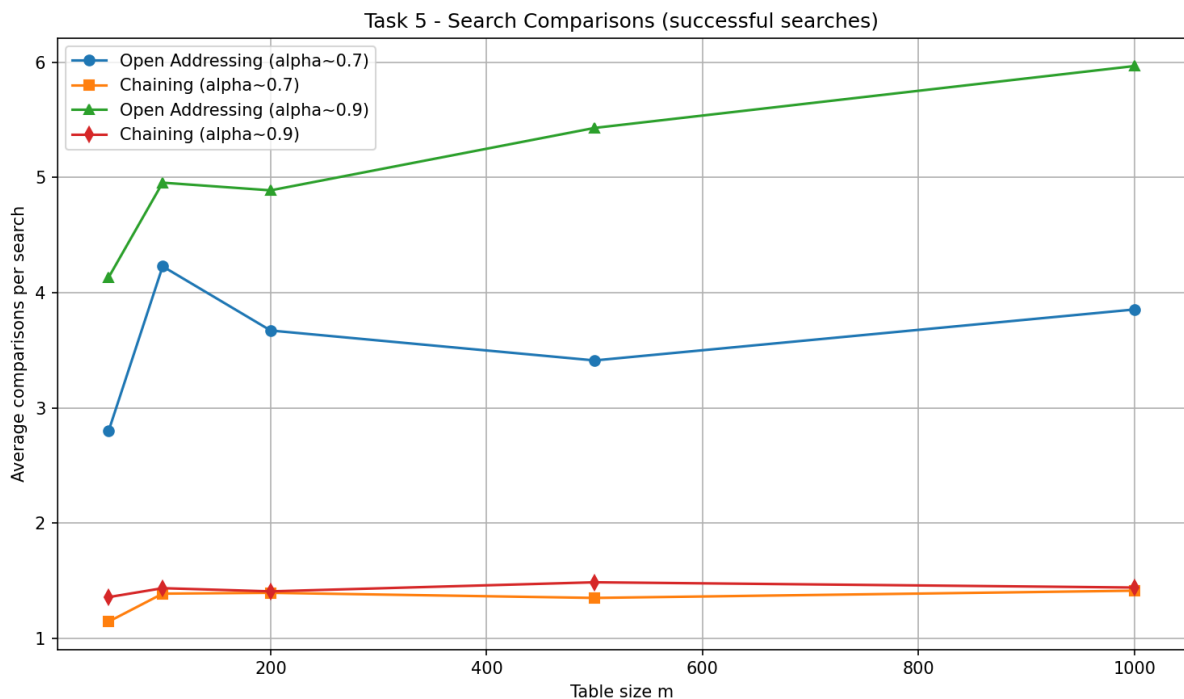
d) Sondierungsanzahl

Sondierungszahl für $\alpha = 0,5 \rightarrow 2,0$

Sondierungszahl für $\alpha = 0,9 \rightarrow 10,0$

Bis zu einem $\alpha = 0,8$ (Sondierungszahl 5,0) ist es in der Praxis noch akzeptabel.

5. Auswertung



a) Mehr vergleiche bei gleichem Alpha

Die offene Adressierung benötigt mehr Vergleiche.

b) Belegungsfaktor 0,9

Die verkettete Liste benötigt nicht viel mehr Vergleiche als bei einem Belegungsfaktor von 0,7.

Bei größeren Tabellengrößen benötigt eine Hashtabelle mit offener Adressierung bei einem Belegungsfaktor von 0,9 knapp zwei Vergleiche Mehr als bei einem Belegungsfaktor von 0,7

c) Vorteile

Verkettete Liste:

Das Einfügen ist auch über einem Alpha von 1 möglich.

Der Löschvorgang ist simpel.

Offene Adressierung:

Kompaktes Speicherlayout (definierter Speicherplatz) und Cache-freundlicher.